Absolutely. Here are **50 more scenario-based GenAI interview questions with answers**, focused on the kind of practical questions a **4+ years GenAI engineer** can get.

■ GenAI Scenario-Based Interview — Set 2

RAG & Retrieval

51. Your RAG retrieves the correct document but the wrong page. What do you check?

Answer:

> "I would check chunk-to-page metadata mapping. Each chunk should retain the document ID and page number during ingestion. I would also inspect the retrieved chunks and similarity scores to verify that the correct page is being ranked."

---

52. Two documents contain exactly the same information. What problem can this cause?

Answer:

> "Duplicate documents can produce duplicate chunks in retrieval and waste context and tokens. I would detect duplicates using document IDs or content hashes and avoid indexing duplicate content."

---

53. A document was deleted, but the chatbot still answers using it. Why?

Answer:

> "The document or its embeddings may still exist in the vector database. I would check whether the corresponding chunks were deleted or marked inactive and verify the index. I would then run a test query to confirm the deleted content is no longer retrieved."

---

54. Users ask questions using different terminology than the documents.

Example:

> User: "How do I reboot a server?"
>
> Document: "Restart procedure"

Answer:

> "This is where semantic embeddings are useful because they capture meaning rather than only exact keywords. If retrieval is still poor, I would consider query expansion, better embeddings, or hybrid search."

---

55. The user asks a very short question like "password?"

Answer:

"The query is ambiguous, so I would use conversation history or ask a clarification question. I would avoid retrieving large amounts of unrelated context based on an ambiguous query."

---

56. User asks a question containing a typo.

"How to restat EC2?"

Answer:

"I would rely on semantic retrieval and possibly query normalization or spelling correction. The system should ideally understand that 'restat' means 'restart' based on context."

---

57. User asks a question containing an exact error code.

"What does ORA-01017 mean?"

Answer:

"I would use hybrid search because exact error codes are better handled by keyword search, while semantic search can retrieve related explanations."

---

58. Retrieval quality drops after changing the embedding model. What do you do?

Answer:

"I would not mix embeddings from incompatible models in the same index. I would create a new index using the new embedding model, evaluate it against the existing test dataset, and switch after validating retrieval quality."

---

59. Can you change the embedding model without reprocessing documents?

Answer:

"Normally, no. If the embedding representation changes, the existing documents need to be re-embedded using the new model so that query and document vectors are in the same vector space."

---

60. How do you choose an embedding model?

Answer:

"I consider semantic retrieval quality, domain relevance, language support, vector dimensions, latency, cost, and deployment requirements. I would validate the model using our actual domain questions rather than selecting it only based on benchmarks."

■ Chunking Scenarios

61. How would you chunk a legal document?

Answer:

"I would prefer structure-aware chunking rather than blindly splitting every fixed number of characters. I would preserve sections, clauses, headings, and paragraph relationships because legal meaning often depends on surrounding context."

---

62. How would you chunk an AWS documentation PDF?

Answer:

"I would preserve headings, service names, procedures, and related steps. I would avoid splitting instructions in the middle of a procedure and use appropriate overlap where necessary."

---

63. Fixed-size chunking is giving poor results. What would you try?

Answer:

"I would try semantic or structure-aware chunking. I would use document headings, paragraphs, sections, tables, and other natural boundaries instead of only character count."

---

64. What happens if chunk overlap is too large?

Answer:

"It creates duplicate information across chunks, increasing storage, retrieval redundancy, and token usage. I would tune overlap based on the document structure."

---

65. What happens if chunk overlap is too small?

Answer:

"Important context can be lost when information spans two chunks. I would increase overlap or use a structure-aware or parent-child retrieval approach."

■ Prompt Engineering

66. The model ignores your instructions. What do you check?

Answer:

"I would check prompt structure, conflicting instructions, context placement, system versus user instructions, and prompt length. I would simplify the prompt and clearly define the expected behavior and output format."

---

67. How would you force the model to return JSON?

Answer:

"I would provide a clear JSON schema or structured output definition and validate the response against that schema. If the response is invalid, I would retry or repair it carefully."

---

68. The model returns extra text along with JSON.

Answer:

"I would use structured output or JSON mode if supported. I would also validate the response programmatically instead of trusting the model's formatting."

---

69. Your prompt is becoming very large. What do you do?

Answer:

"I would remove redundant instructions, reduce retrieved context, summarize conversation history, use relevant chunks only, and avoid sending unnecessary metadata."

---

70. How do you version prompts?

Answer:

"I would store prompts in version control or a prompt management system. Each version should have an identifier, test results, and deployment history so we can roll back if a new prompt reduces quality."

■ LLM Behavior

71. Temperature is already low but answers are still inconsistent. Why?

Answer:

"I would check whether the retrieved context is changing between requests, whether conversation history is changing, and whether other sampling parameters or model behavior are contributing. Deterministic output also depends on

consistent inputs."

---

72. What happens if temperature is too high for an enterprise chatbot?

Answer:

"The responses can become more variable and creative, which may increase the risk of unsupported information. For factual enterprise Q&A, I would generally use a lower temperature."

---

73. When would you use a higher temperature?

Answer:

"For creative tasks such as brainstorming, marketing copy, storytelling, or generating multiple ideas. I would use lower randomness for factual or deterministic tasks."

---

74. One model is accurate but expensive. Another is cheaper but slightly less accurate. Which do you choose?

Answer:

"I would evaluate them against our business requirements. I could use a model-routing approach where simple tasks use the cheaper model and complex or high-risk tasks use the more capable model."

---

75. How would you implement model routing?

Answer:

User Request

↓

Classifier / Router

↓

■■■■■■■■■■■■■■■■■■■■■■■■

Simple Complex

↓ ↓

Small Large

Model Model

"This can reduce cost while maintaining quality for complex requests."

■ Conversation & Memory

76. The user has a 50-message conversation. Sending all history is expensive. What do you do?

Answer:

"I would not send the entire conversation every time. I would keep recent messages, summarize older messages, and retrieve only relevant historical information when required."

---

77. What is conversational memory?

Answer:

"Conversational memory allows the system to retain relevant information from previous interactions so that future questions can use that context."

---

78. User asks:

"What about the previous policy?"

Answer:

"I would use conversation history to resolve what 'previous policy' refers to. If the reference remains ambiguous, I would ask the user for clarification rather than retrieving unrelated documents."

---

79. User changes the topic completely. Should you send the entire previous conversation?

Answer:

"No. I would identify the new topic and avoid sending irrelevant history. This reduces context size and prevents old information from influencing the new answer."

■ Security

80. How do you protect API keys?

Answer:

"I would never hard-code API keys in source code. I would use environment variables or a secrets manager, restrict permissions, rotate credentials, and prevent secrets from appearing in logs."

---

81. A developer accidentally commits an API key to Git. What do you do?

Answer:

"I would immediately revoke or rotate the exposed key, remove it from active use, check access logs, and clean the repository history where appropriate. Then I would move the credential to a secure secret-management solution."

---

82. How do you secure a GenAI API?

Answer:

"I would use authentication, authorization, HTTPS, rate limiting, input validation, secrets management, logging, monitoring, and appropriate access controls."

---

83. How do you prevent users from accessing another customer's data?

Answer:

"I would enforce tenant isolation and authorization before retrieval. Every document and chunk would contain tenant or access metadata, and the retrieval query would filter using the authenticated user's permissions."

---

84. What is multi-tenant RAG?

Answer:

"It is a RAG system serving multiple customers or organizations while keeping their data isolated. Tenant ID must be part of the access-control and retrieval filtering strategy."

■ Production Reliability

85. What is a circuit breaker?

Answer:

"A circuit breaker temporarily stops requests to a failing downstream service. This prevents repeated failed calls and allows the service time to recover."

---

86. What is graceful degradation?

Answer:

"Graceful degradation means the application continues providing a useful service even when one component is unavailable. For example, if the LLM is temporarily unavailable, we can show cached answers or a controlled error

instead of crashing the entire application."

---

87. What is a health check?

Answer:

"A health check verifies whether a service or dependency is available and functioning. For a GenAI application, I could monitor the API server, vector database, Redis, and LLM provider."

---

88. Your application has memory leaks. How do you troubleshoot?

Answer:

"I would monitor memory usage over time, inspect application logs and metrics, check object/resource lifecycle, and profile the application. I would also check whether large documents, embeddings, or cached objects are being retained unnecessarily."

---

89. Your application crashes when processing large PDFs.

Answer:

"I would avoid loading the entire document into memory. I would process documents in batches or streams, split processing into smaller units, use asynchronous workers, and monitor memory usage."

■ Deployment & Scaling

90. How would you deploy a GenAI application?

Answer:

"I would containerize the application using Docker and deploy it behind a load balancer. I would use auto-scaling, centralized logging, monitoring, secrets management, health checks, and CI/CD for controlled deployments."

---

91. Why should the RAG application be stateless?

Answer:

"A stateless application allows any instance to handle a request. This makes horizontal scaling easier. Conversation state or cache data can be stored externally in systems such as Redis or a database."

---

92. What happens if one application instance crashes?

Answer:

"The load balancer should stop sending traffic to the unhealthy instance. Auto-scaling or orchestration should replace it, while other healthy instances continue serving users."

---

93. How would you deploy a new LLM version safely?

Answer:

"I would test it against an evaluation dataset first, then use a controlled rollout such as canary or A/B deployment. I would monitor quality, latency, errors, and cost before fully switching traffic."

■ Data & Document Pipeline

94. A PDF contains images instead of text. How do you process it?

Answer:

"I would use OCR or a multimodal document extraction solution to extract the text and relevant information from the images. I would validate the extracted content before indexing it."

---

95. A document contains confidential information mixed with public information. What do you do?

Answer:

"I would classify the content and apply access control at the chunk or document level where necessary. Sensitive information can also be masked or removed before embedding."

---

96. How do you know whether a document was successfully indexed?

Answer:

"I would track an ingestion status such as uploaded, extracted, chunked, embedded, indexed, or failed. I would also store document IDs and verify that the expected number of chunks and embeddings were created."

---

97. Embedding generation fails for 100 documents out of 10,000. Do you restart everything?

Answer:

"No. I would use checkpointing and retry only the failed documents. This avoids reprocessing successful documents and reduces processing time and cost."

---

98. How would you handle a document ingestion failure?

Answer:

"I would capture the document ID and failure reason, retry transient failures, move persistent failures to a dead-letter or error queue, and provide monitoring and alerts."

■ Advanced Architecture

99. When would you use RAG + fine-tuning together?

Answer:

"Fine-tuning can teach the model a specific behavior, style, or task pattern, while RAG provides current external knowledge. For example, I could fine-tune the response style and use RAG to retrieve frequently changing company policies."

---

100. When would you NOT use RAG?

Answer:

"If the task doesn't require external knowledge, RAG may add unnecessary complexity. For example, simple text transformation, summarization of user-provided text, or creative generation may not require a vector database."

■ 10 Very Important "What Would You Do?" Questions

These are worth practicing repeatedly:

| ScenarioFirst thing to think about | |

| ---------- | ---------- |

| Wrong answer | **Check retrieval** |

| No answer in documents | **Similarity threshold + I don't know** |

| Too much context | **Top-K + reranking** |

| High latency | **Measure each stage** |

| High cost | **Tokens + number of LLM calls** |

| 429 errors | **Rate limit + backoff** |

| 10K users | **Load balancer + auto-scaling** |

| Confidential data | **Authorization before retrieval** |

| Old document | **Version + metadata** |

| Hallucination | **Groundedness + prompt + evaluation** |

One formula to remember for interviews

Good RAG =

> **Good Documents + Good Chunking + Good Embeddings + Good Retrieval + Good Prompt + Guardrails + Evaluation + Monitoring**

If you can explain **why each component is needed and what you would check when it fails**, you're moving from a basic RAG answer toward a **4+ years production-level answer**.